C# CHAPTER 3

Practical Assignment

Classes and Object-Oriented Programming

Instructions

- Create a separate C# program for each question.
- Use classes and objects wherever required.
- Apply fields, properties, constructors, and methods as specified.
- Use appropriate access modifiers such as public and private.
- Demonstrate the output by creating objects and calling methods.

Section 1 - Classes, Fields, Properties and Methods

1. Car Management

- Create a Car class with a private field to store the car model.
- Create public auto-properties for Color and Year.
- Create a Model property with explicit get and set accessors.
- Create a constructor to initialize the car model, color, and year.
- Use the this keyword wherever required.
- Create a DisplayDetails() method to display the car information.
- Create two Car objects with different details and display their information.

2. Student Information

- Create a Student class with Name, RegisterNumber, and Department properties.
- Create a constructor to initialize all three properties.
- Create a DisplayDetails() method to display student information.
- Create at least three Student objects with different information.

3. Employee Details

- Create an Employee class with a private field to store the employee's salary.
- Create public properties for EmployeeId and Name.
- Create a property to read and update the salary.
- Create a constructor to initialize employee details.
- Create a DisplayDetails() method to display employee information.
- Create two Employee objects and display their details.

Section 2 - Static Methods

4. Calculator Using Static Methods

- Create a Calculator class with the following static methods:
- Add(int a, int b) - returns the sum of two numbers.
- Subtract(int a, int b) - returns the difference between two numbers.
- Multiply(int a, int b) - returns the product of two numbers.
- Divide(double a, double b) - returns the division result.
- Call all methods using the class name without creating a Calculator object. Display the results.

5. Temperature Converter

- Create a TemperatureConverter class with two static methods:
- CelsiusToFahrenheit(double celsius) - converts Celsius into Fahrenheit.
- FahrenheitToCelsius(double fahrenheit) - converts Fahrenheit into Celsius.
- Call both methods directly using the class name and display the converted temperatures.

Section 3 - Constructor Overloading

6. Product Management

- Create a Product class with Name, Price, and Stock properties.
- Implement a parameterless constructor that initializes the product with default values.
- Implement a two-parameter constructor that accepts the product name and price. Initialize stock to 10.
- Implement a three-parameter constructor that accepts the product name, price, and stock.
- Create a DisplayInfo() method to display the product details.
- Create three Product objects using the three different constructors and display their information.

7. Book Information

- Create a Book class with Title, Author, and Price properties.
- Implement a parameterless constructor that assigns default values.
- Implement a constructor that accepts the title and author.
- Implement a constructor that accepts the title, author, and price.
- Create a DisplayBookDetails() method to display the book information.
- Create objects using all three constructors and display their details.

Section 4 - Method Overloading

8. Area Calculator

- Create an AreaCalculator class with overloaded methods named CalculateArea().
- CalculateArea(int length, int breadth) - calculates the area of a rectangle.
- CalculateArea(int side) - calculates the area of a square.
- CalculateArea(double radius) - calculates the area of a circle.
- Call each overloaded method with appropriate arguments and display the results.

9. Log Management

- Create a LogManager class with two overloaded methods named DisplayLog().
- DisplayLog(string message, int code) - displays the log message along with its code.
- DisplayLog(int code, string message) - displays the code followed by the log message.
- Call both methods using different argument orders and observe which method gets executed.

Section 5 - Access Modifiers and Encapsulation

10. Bank Account Management

- Create a BankAccount class with a private field named balance.
- Create a public property named AccountHolder.
- Create a public property named AccountNumber with a private setter.
- Create a public Balance property that allows reading the balance but does not allow direct modification.
- Create a constructor to initialize the account holder, account number, and initial balance.
- Create a Deposit() method to add money to the account.
- Create a Withdraw() method to withdraw money from the account.
- Create a DisplayAccountDetails() method to display the account information.
- Create a BankAccount object and demonstrate depositing, withdrawing, and displaying the updated balance.